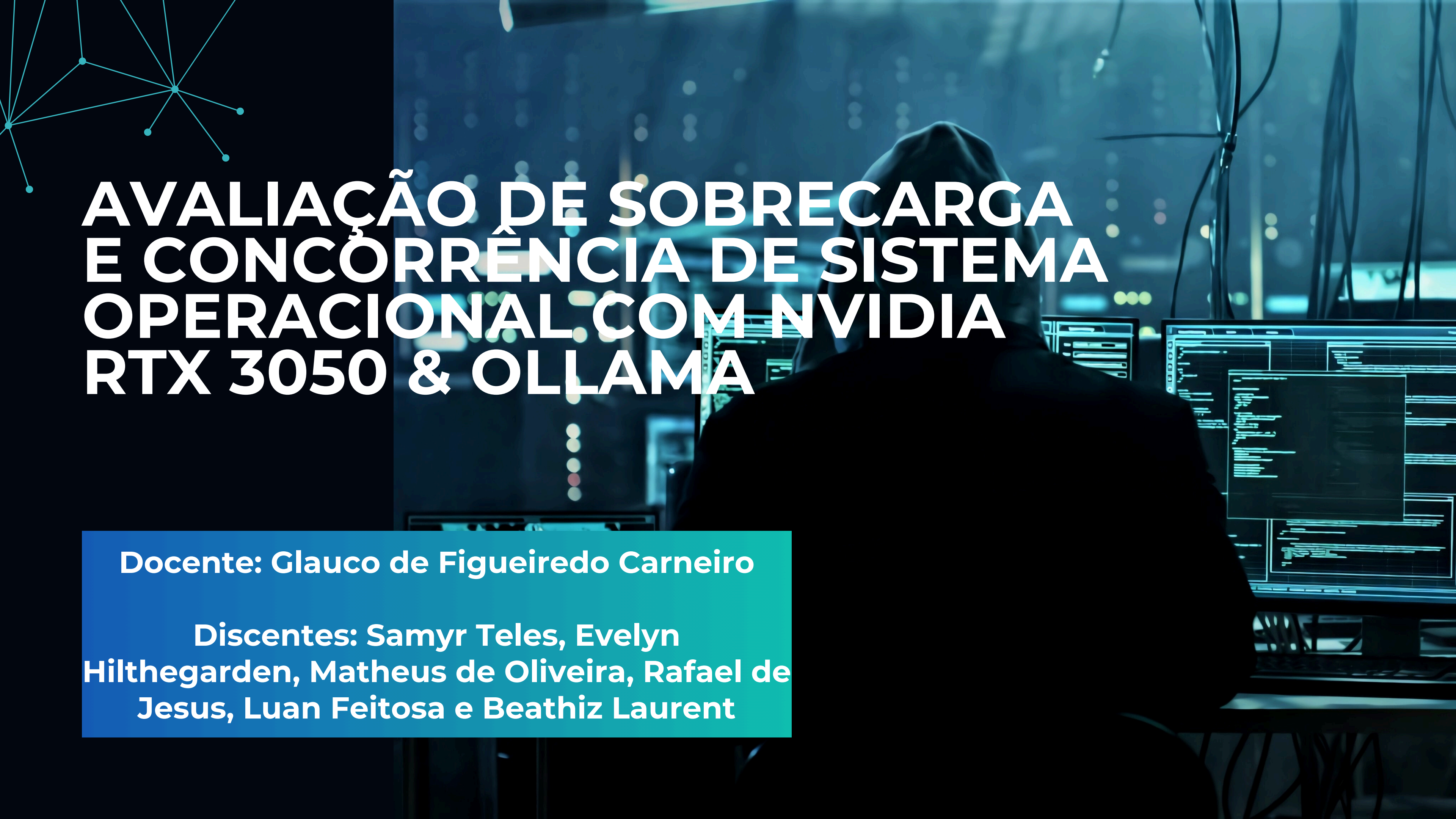




AVALIAÇÃO DE SOBRECARGA E CONCÓRRENCIA DE SISTEMA OPERACIONAL COM NVIDIA RTX 3050 & OLLAMA

Docente: Glauco de Figueiredo Carneiro

**Discentes: Samyr Teles, Evelyn
Hilthegarden, Matheus de Oliveira, Rafael de
Jesus, Luan Feitosa e Beathiz Laurent**

Trilha de Aplicação & Contexto Integrado

Arquitetura Selecionada

Trilha A do projeto: Integração do engine de IA Ollama com a interface web Open WebUI executada via contêiner Docker.

- **Interface:** Open WebUI em Container isolado.
- **Runtime LLM:** Ollama nativo/host gerenciando modelos localmente.
- **Modelo:** Qwen2.5-1.5B-Instruct quantizado em Q4_K_M.

Relevância para a Disciplina de SO

A trilha permite a análise prática e integrada de múltiplos conceitos essenciais de Sistemas Operacionais:

- **Threads & Concorrência:** Escalonamento e alocação de threads nativas do SO.
- **Comunicação Interprocessos:** Sockets TCP e HTTP via API REST local (porta 11434).
- **Isolamento (Docker):** Containers, namespaces, cgroups e mapeamento de volumes.
- **Perfis de Memória:** Consumo de RAM vs. VRAM e comportamento do KV Cache.

Seleção do Modelo LLM & Parâmetros

PARÂMETROS TOTAIS

1.5B

Adequado ao limite estabelecido (≤ 10 Bilhões).

FORMATO / QUANTIZAÇÃO

Q4_K_M

Formato quantizado GGUF leve via llama.cpp.

CONSUMO BASE EM DISCO

~935–986 MB

Baixa pegada de memória RAM/VRAM estática.

Motivação da Escolha

O modelo Qwen2.5-1.5B-Instruct foi selecionado por sua extrema eficiência computacional. Como seu footprint inicial é de apenas ~0.9 GB, ele possibilita a realização de testes rigorosos de concorrência e requisições simultâneas, inclusive em computadores sem placa gráfica dedicada, sem atingir a exaustão de memória da máquina hospedeira.

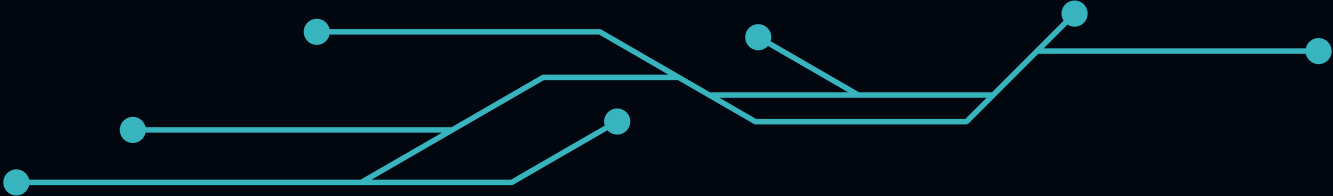
Ambientes Experimentais de Hardware

Ambiente 1: Com GPU DEDICADA

- SO: Linux (Ubuntu 24.04.2 LTS)
- CPU: AMD Ryzen 5 5600GT (6 núcleos / 12 threads)
- GPU: NVIDIA GeForce RTX 3050 (6144 MiB VRAM)
- RAM Mapeada (WSL): 15 GB Total
- Ollama Runtime: Versão 0.33.3 (Aceleração CUDA)

Ambiente 2: Sem GPU (CPU-Only / WSL2)

- SO: Ubuntu via WSL2 (Kernel 6.6.114.1-microsoft)
- CPU: AMD Ryzen 5 5500 (6 núcleos / 12 threads)
- GPU Física: AMD RX 7600 (8GB VRAM)
- Limitação Técnica: Incompatibilidade de drivers AMD no WSL2; SO detectou apenas o *Microsoft Basic Render Driver*.
- Execução: 100% CPU via instrução host.



MÉTRICAS DA EXECUÇÃO PADRÃO

118.5
Tokens / Segundo

97%
Uso máximo de GPU

59.5W
Consumo Máximo



Repetibilidade do Teste Padrão

Teste	Tempo Total	Tokens Gerados	Velocidade	Temp. Máx	VRAM Alocada
A-1	3.53 s	415 tok	118.54 tok/s	56 °C	1982 MiB
A-2	3.51 s	415 tok	118.91 tok/s	56 °C	1965 MiB
A-3	3.54 s	415 tok	118.12 tok/s	57 °C	1965 MiB

O SO manteve estabilidade determinística perfeita com variação menor que 0.6% no tempo de geração entre as execuções.



METODOLOGIA & AMBIENTE

Testes executados em ambiente Linux nativo (sem VM/Contêineres) para isolamento total do SO.

Impacto de Parâmetros em VRAM & RAM

Teste A & B (Contexto 4096 Tokens)

O modelo manteve alocação estática de VRAM constante durante o processamento.

ALOCAÇÃO VRAM

1.2 GB

TEMPO CARREGAMENTO

0.0006s (A) / 1.96s (B)

Teste C (Contexto 8192 Tokens)

Ao dobrar a janela de contexto de tokens, o KV Cache alocado expandiu significativamente.

ALOCAÇÃO VRAM

1.4 GB (+200 MB)

TEMPO CARREGAMENTO

3.3198 s

Impacto na RAM Global do Sistema: No ambiente com GPU, a RAM iniciou em 8.5 GiB e subiu para 9.2 GiB durante a inferência (acréscimo de **0.7 GiB / 700 MiB**). Esse espaço suporta as requisições HTTP do Ollama daemon e buffers de I/O de rede.

TESTE DE CONCORRÊNCIA (CONFIG B)

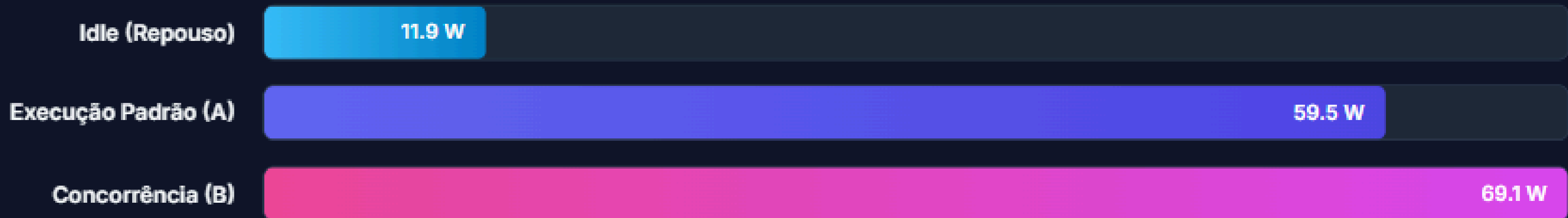


Carga Simultânea no SO

Foram disparadas 3 requisições paralelas (Req 0, Req 1, Req 2) para avaliar o escalonamento do Kernel Linux e do gerenciador Ollama. O consumo elétrico subiu para 69.12W e a temperatura atingiu 59°C, demonstrando carga máxima nos núcleos Tensor e CUDA.



Comparação de Potência Elétrica (W)



A concorrência forçou a GPU perto do seu TDP máximo na RTX 3050 mobile/desktop, exigindo +16% de energia total.

Mapeamento de Processos: Ambiente GPU

⚙️ Processo de Computação Dedicado

A inferência executou de forma isolada via backend llama-server nativo:

```
> llama-server (Ollama Compute) PID 78507
```

- Tipo de Carga: Compute (Tipo C no *nvidia-smi*).
- Alocação de VRAM: Estável em 1208 MiB nos 3 testes.
- Comunicação: Resposta direta à API local.

🖥️ Concorrência com Sistema Operacional

A GPU dividiu recursos dinamicamente com os processos gráficos (Tipo G):

- Xorg (PID 2761) & gnome-shell (PID 2965) : Interface visual.
- nautilus (PID 76850) : Gerenciador de arquivos.
- steamwebhelper (PIDs 6864, 7086) : Processos em segundo plano.

🔧 Mapeamento de Processos: Ambiente CPU

🏗️ Daemon Principal & Threads

Devido à ausência de driver GPU no WSL2, o Ollama paralelizou o modelo em CPU:

🐧 `/usr/local/bin/ollama serve` **PID 1294**

☰ **Threads de Inferência (15 simultâneas)** **PIDs 1381-1365**

As 15 threads dividiram o cálculo vetorial sobre as 12 threads lógicas do Ryzen 5 5500.

🧠 Sobrecarga de Recursos

- **Uso de CPU:** Atingiu **100% constante** durante a inferência.
- **RAM Alocada:** Todo o peso de 1.2 GB e contexto foram mantidos estritamente na RAM (4.0 GiB em uso, 14 GiB disponíveis).
- **Gargalo:** Ausência de instrução VRAM forçou estrangulamento da CPU hospedeira.

</> Parentesco & Chamadas de Sistema (Syscalls)

👤 Hierarquia Pai e Filho

- **Processo Pai (PID 1294):** Possui `PPID = 1` (adotado pelo `init/systemd`). Atua como escutador HTTP na porta 11434 e orquestrador de tarefas.
- **Processos Filhos / Threads:** Executam os cálculos matemáticos concorrentes.

📄 Perfil do `strace` (Ambiente CPU)

Syscall	% Tempo	Chamadas	Responsabilidade
<code>futex</code>	88.29%	42,924	Sincronização entre 15 threads
<code>accept4</code>	5.41%	89	Recepção de conexões HTTP da API
<code>read / write</code>	2.26%	11,489	Tráfego de tokens e I/O de disco

Comparativo de Ambientes Experimentais

Critério	Ambiente Com GPU (NVIDIA RTX 3050)	Ambiente Sem GPU (CPU / WSL2 AMD)
Componente Principal	GPU NVIDIA RTX 3050 (6144 MIB VRAM)	AMD Ryzen 5 5500 (6 Cores / 12 Threads)
Carga de Processamento	GPU dedicada (93% a 97% na Inferência)	100% CPU compartilhada entre threads
Alocação do Modelo	Totalmente em VRAM (1208 MIB Isolados)	Alocado na Memória RAM principal do SO
Paralelismo	Subprocesso Compute <code>llama-server</code>	Daemon <code>ollama serve</code> com ≥ 15 threads
Impacto no Hospedeiro	Mínimo em CPU; rápida desocupação (1% Idle)	Sobrecarrega processadores e trava recursos



GARGALOS DE SISTEMA OPERACIONAL

Análise de chaveamento de contexto,
Filas de Requisições e Gerenciamento de
Memória VRAM.

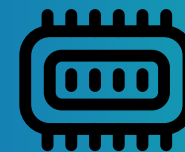


IMPACTO DO ESCALONAMENTO



Fila Batching

O Ollama enfileirou as requisições mantendo a taxa individual próxima a ~119 tok/s, mas aumentando a latência total observada pelo cliente.



Alocação VRAM

A alocação de memória do llama-server permaneceu eficiente em ~1.2 GB VRAM, evitando trocas excessivas com a memória RAM via PCIe.



Thermal Headroom

O SO e a GPU gerenciaram a curva térmica até 59°C sem acionar o thermal throttling (estrangulamento térmico por hardware).

Ciclo de Vida & Utilização da GPU

1. REPOUSO (IDLE)

1%–5%

Modelo retido na VRAM sem carga ativa.

2. INICIALIZAÇÃO

~6%

Alocação rápida de buffers e contexto.

3. GERAÇÃO (TOKENS)

93%–97%

Pico de computação paralela na GPU.

4. RETORNO A IDLE

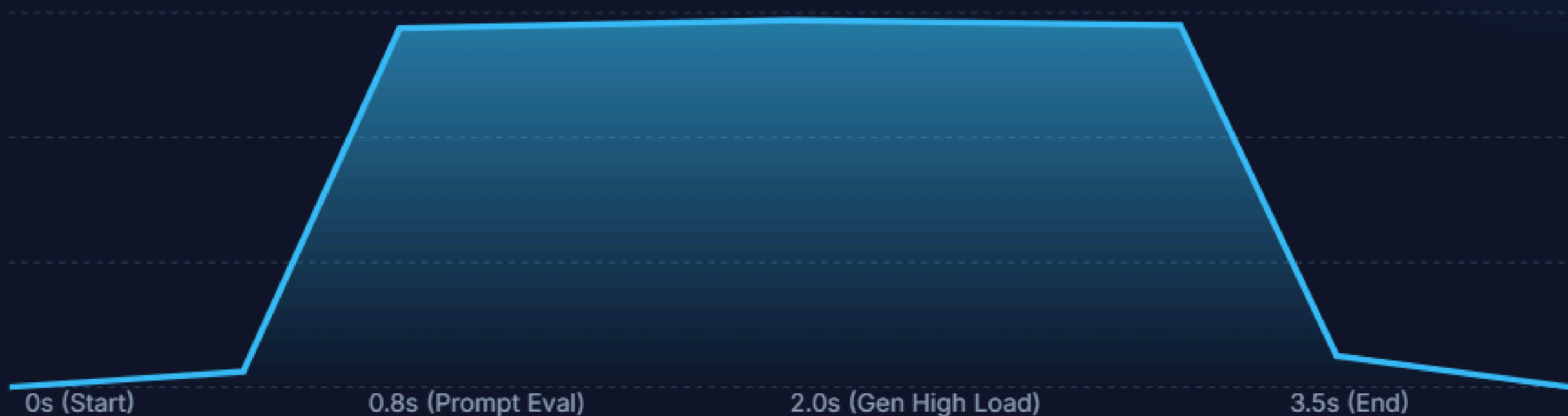
1%

Queda imediata após a resposta.

Eficiência Energética e Térmica

O ciclo acima demonstra que o descarregamento da computação na GPU não apenas libera a CPU para tarefas de SO, mas também otimiza a dissipação térmica e reduz o consumo elétrico, operando em pico somente durante o streaming de tokens.

Evolução Temporal de Uso de GPU (%)



Pico súbito de 1% para 97% durante a fase de Token Generation, demonstrando rápida prontidão do driver NVIDIA no Linux.



PRINCIPAIS CONCLUSÕES



Eficiência Linux Nativo:

Mapeamento direto no driver NVIDIA reduz latência de chamadas do sistema (syscalls).



Capacidade da RTX 3050 (6GB):

Ideal para modelos quantizados de 1.5B parâmetros mantendo throughput superior a 115 tok/s.



Comportamento sob Carga:

O sistema prioriza a vazão (throughput) por requisição em vez de distribuição equilibrada em tempo real.

Conclusões do Estudo Prático

Arquitetura de SO

A separação entre Daemon (API HTTP), Container UI (Open WebUI) e Backend de Inferência (`llama-server`) comprova a eficiência do modelo cliente-servidor em ambiente de IA.

Impacto de Tokens

O aumento da janela de contexto (4k → 8k) provoca crescimento linear no KV Cache na VRAM (+200MB) e exige um custo de carregamento 5x maior (0.6ms → 3.3s).

Gargalo de Driver

A falta de suporte a GPU no WSL2 impõe severa penalidade, forçando 88.29% do tempo da CPU em chamadas `futex` para sincronizar threads de computação.



OBRIGADO!!!